

JSON to TypeScript – Simple Guide

Tool Name

JSON to TypeScript

What This Tool Does

JSON to TypeScript is a developer tool that **converts JSON data into TypeScript interfaces or types** automatically. It helps you quickly create **strongly typed models** for your frontend or backend projects.

You can also control whether properties are **optional or required**.

Who Can Use This Tool

- Frontend developers (React, Vue, Svelte, Angular)
 - Backend developers (Node.js, NestJS)
 - TypeScript learners
 - API consumers
 - Anyone working with JSON data
-

Main Features

- Convert JSON to TypeScript **interface** or **type**
 - Generate nested interfaces automatically
 - Per-property optional control
 - Make all fields optional or required
 - Copy generated `.ts` code
 - Clean and readable output
-

Tool Options Explained

- **Interface** – Generates TypeScript interfaces
 - **Type** – Generates TypeScript type aliases
 - **All Optional** – Makes all properties optional (?)
 - **All Required** – Makes all properties required
 - **Copy** – Copies the generated TypeScript code
 - **.ts** – Output is ready to use in a TypeScript file
-

How to Convert JSON to TypeScript

Step 1: Open the Tool

Open the **JSON to TypeScript** tool.

Step 2: Paste JSON Data

Paste your JSON into the **Input JSON** editor.

Example JSON:

```
{
  "id": 1,
  "name": "John Doe",
  "email": "john@example.com",
  "address": {
    "street": "123 Main St",
    "city": "New York"
  },
  "tags": ["developer", "svelte"]
}
```

Step 3: Choose Output Type

Select one option: - **Interface** - **Type**

Step 4: Set Property Rules

- Choose **All Optional** if fields may be missing
- Choose **All Required** if all fields must exist

Step 5: Generate Code

The tool will instantly generate TypeScript code.

Example Output

```
interface RootObject {
  id: number;
  name: string;
  email: string;
  address: Address;
```

```
    tags: string[];
  }

  interface Address {
    street: string;
    city: string;
  }
```

Common Use Cases

- Create TypeScript models from API responses
- Convert backend JSON to frontend types
- Improve type safety in projects
- Speed up development
- Avoid manual typing errors

Benefits

- Saves development time
- Reduces human errors
- Improves code readability
- Makes TypeScript projects safer

Tips

- Make optional fields when API data is inconsistent
- Rename interfaces after generation if needed
- Keep one interface per file for large projects
- Always validate JSON before converting